SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 09

Introduction to Flutter and Mobile Application Development



Year 3 • Semester 1 • 4 Credits • Duration: 3 Hours

Learning Outcomes: LO1, LO2, LO4 | Mobile App Development Frameworks

Where This Lecture Fits

L01	L02	L03	L04	L05	L06	L07	L08	L09	L10	L11

11 lectures — today we zoom into mobile development

This module section

Mobile App Development Frameworks

15 contact hours • spans Lectures 09 – 10

- Mobile application architecture across platforms
- Building cross-platform apps — introduction to Flutter
- Accessing device features, handling UI, managing data (Lecture 10)

In the next 3 hours you will learn

- Native, cross-platform, hybrid & PWA approaches
- What Flutter is and how it draws every pixel
- The Dart language — null safety & async basics
- Widgets: the building blocks of every screen
- State, events & navigation between screens
- Layered mobile architecture for maintainable apps
- Building “TaskFlow” — a complete cross-platform app

Learning Outcomes for This Lecture

LO1

Evaluate frameworks

Evaluate the types of mobile frameworks used in modern software — native, cross-platform, hybrid and PWA.

LO2

Apply & build

Apply Flutter, Dart and widget-based design to build a working cross-platform mobile application.

LO4

Select & justify

Select and justify an appropriate mobile framework and architecture for given project requirements.

By the end of today you will be able to…

- Explain how cross-platform frameworks build one codebase for Android and iOS, and when to choose them.
- Read and write basic Dart, compose widgets into screens, manage state with setState(), and navigate between screens.
- Describe a layered mobile architecture and structure a small Flutter project the way a professional team would.

Why Mobile Development — and Why Flutter?

Why mobile matters

Mobile-first world

Most digital traffic and many products are now phone-first, not desktop-first.

Always-on engagement

Push notifications, location and the camera make apps central to modern systems.

Business demand

Banking, health, retail and logistics all ship mobile apps — strong job market.

Why Flutter for this module

- **One codebase → Android, iOS, web & desktop**
- Backed by Google, free and open-source
- **Fast UI with a rich widget library**
- Hot reload makes the build-see loop almost instant
- **Great fit for full-stack student projects**
- Pairs naturally with the REST APIs you built earlier in SE3090
- **In-demand skill for internships and industry**

Today’s Roadmap — 3 Hours

Hour 1

Foundations & frameworks

Mobile approaches (native / cross-platform / hybrid / PWA) • framework landscape • cross-platform development • selection criteria • Checkpoint 1

Hour 2

Flutter, Dart & widgets

What Flutter is & its architecture • Dart basics, null safety, async • project structure • widgets, layout, lists, input • reusable widgets • Activity

Hour 3

State, navigation, architecture & build

setState & events • forms & validation • navigation • layered architecture • building TaskFlow end-to-end • mistakes • Quiz

Short breaks after each hour — ask questions at any time.

PART 1 OF 6

Mobile Development Foundations



- What mobile application development really involves
- Four ways to build: native, cross-platform, hybrid, PWA
- The framework landscape and the Android / iOS ecosystems
- Benefits, limitations and how teams choose a framework

What Is Mobile Application Development?

Definition. Designing, building, testing and maintaining software that runs on mobile devices — smartphones and tablets — making use of their screen, touch input, sensors, camera, location and connectivity.

Role in modern software systems

The front door

Often the main way users reach a system — the client for your backend APIs.

Device-aware

Uses GPS, camera, notifications and offline storage that web apps cannot easily reach.

Connected

Talks to REST services, databases and cloud — part of a larger full-stack system.

Experience-critical

Performance, responsiveness and offline behaviour shape how users judge the product.

Four Ways to Build a Mobile App



Native

Kotlin/Java • Swift

- Built per-platform with the vendor SDK
- Best performance & full device access
- Two codebases → double the work



Cross-platform

Flutter • React Native

- One codebase → Android + iOS
- Near-native performance
- Today's focus — best balance



Hybrid

Ionic • Cordova

- Web tech (HTML/CSS/JS) in a shell
- Fast to reuse web skills
- Heavier, less native feel



Progressive Web App

Installable website

- Runs in the browser, installable
- No app store needed
- Limited device access

Comparing the Four Approaches

Criteria	Native	Cross-platform	Hybrid	PWA
Codebases	One per platform	Single	Single	Single
Performance	Highest	Near-native	Moderate	Browser-bound
Device access	Full	Very good	Limited	Limited
Dev speed / cost	Slowest • high	Fast • low	Fast • low	Fastest • lowest
App store	Yes	Yes	Yes	No (web)
Best for	Performance / HW-heavy	Most apps, MVPs	Content + web reuse	Reach, light apps

Rule of thumb: start cross-platform; go native only when a hard requirement (performance or deep hardware) forces it.

Frameworks & the Android / iOS Ecosystems

Android ecosystem

~70% of phones worldwide

- Language: Kotlin (or Java)
- Tools: Android Studio, Gradle
- Store: Google Play — open distribution

iOS ecosystem

Premium, tightly controlled

- Language: Swift (or Objective-C)
- Tools: Xcode — macOS only
- Store: App Store — strict review

Common mobile development frameworks

Flutter

Dart • Google

React Native

JS • Meta

.NET MAUI

C# • Microsoft

Kotlin MP

Kotlin • JetBrains

Ionic

Web • hybrid

Benefits, Limitations & Selection Factors

Benefits of using a framework

- Reusable UI components & proven patterns
- Faster delivery — less boilerplate
- Large community, packages & documentation
- Cross-platform reach from one codebase

Limitations to watch

- Abstraction can hide platform details
- Large apps & bigger binary size
- New platform features may arrive late
- Learning curve for the framework itself

Factors that drive framework selection

Project requirements	**Performance needs**	**Team skills**	**Timeline & budget**	**Device features**	**Long-term support**

PART 2 OF 6

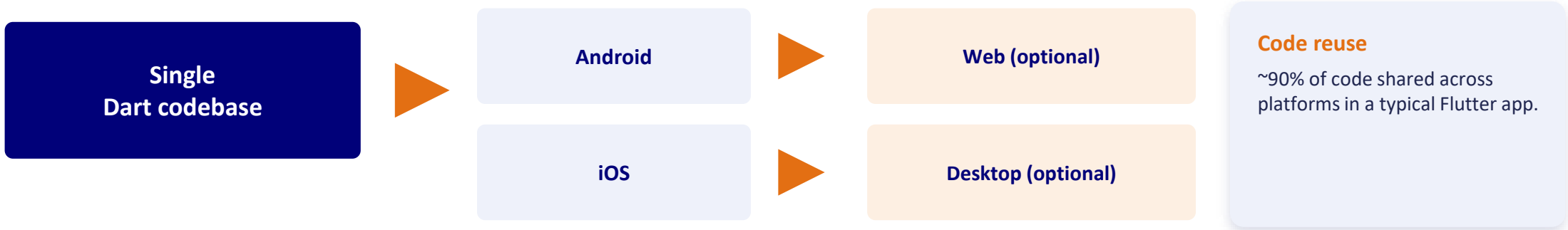
Cross-Platform Development

- What 'single codebase' really means in practice
- The benefits — and the honest trade-offs
- The landscape: Flutter, React Native, MAUI, KMP, Ionic
- Criteria and a simple way to choose



What Is Cross-Platform Development?

Idea. Write your application logic and UI once in a single codebase, then compile or run it on multiple platforms — Android and iOS (and often web and desktop) — instead of building a separate app for each.



Code reuse

~90% of code shared across platforms in a typical Flutter app.

- One language, one team, one set of tests — not two.
- Platform-specific tweaks are still possible where needed.
- Faster releases: fix a bug once and it ships everywhere.

Benefits & Trade-offs of Cross-Platform



Lower cost

One team and codebase instead of two.



Faster to market

Ship to both stores from a single build.



Consistent UX

Same look and behaviour on every device.



Easier maintenance

Fix and test once; everyone benefits.

Trade-offs to plan for — performance & maintenance considerations

- Performance: excellent for typical apps; very heavy graphics or compute-bound games may still favour native.
- Binary size: cross-platform apps ship a runtime, so installs are a little larger.
- Platform parity: a brand-new OS feature can take time to reach the framework — plan around release timing.
- Native escape hatch: when you truly need it, you can call platform code — but that adds per-platform work.

The Cross-Platform Landscape

Framework	Language	Backed by	Strengths	Typical use
Flutter	Dart	Google	Fast UI, one widget set, great tooling	Most new cross-platform apps
React Native	JavaScript	Meta	Reuse React skills, huge ecosystem	Web teams going mobile
.NET MAUI	C#	Microsoft	Native controls, .NET integration	Enterprise / Microsoft shops
Kotlin Multiplatform	Kotlin	JetBrains	Share logic, native UI per platform	Android-first teams
Ionic	HTML/CSS/JS	Ionic	Web tech, quick hybrid apps	Content apps, web reuse

This module: Flutter — a single, consistent widget toolkit and excellent developer experience.

Choosing a Mobile Framework

Requirements

Features, performance & device access the app must have.

Team skills

What languages your developers already know well.

Time & budget

Deadline pressure and available resources.

Support & longevity

Community size, backing and update cadence.

A simple decision flow

Need extreme perf or deep HW?

→ Native

Strong web/React team & deadline?

→ React Native

Want one UI, top tooling?

→ Flutter

Mostly content / web reuse?

→ PWA / Ionic

Checkpoint 1 — Match the Scenario



In pairs (4 minutes): which approach or framework fits each scenario, and why? Match to native, cross-platform (name one), hybrid or PWA.

1. A startup needs an MVP on both stores in 6 weeks with a small team.
2. A 3D-heavy mobile game that must squeeze every frame from the GPU.
3. A university wants a lightweight, installable notice-board reachable from any browser.
4. A company with a strong C#/.NET team wants an internal staff app.
5. An existing React web team must ship a mobile version of their product quickly.

Pair up — then we compare answers as a class.

PART 3 OF 6

Flutter & Dart Foundations



- What Flutter is and how it renders every pixel
- Hot reload, architecture, advantages and limits
- The Dart language: variables, control flow, classes
- Null safety and asynchronous programming basics

What Is Flutter?

Flutter is Google's open-source UI toolkit for building natively-compiled apps for mobile, web and desktop from a single Dart codebase.



Open-source • Google

Stable, widely adopted in industry

Draws its own UI

Flutter paints every pixel with its engine instead of wrapping native controls — identical look on every platform.

Everything is a widget

Screens are built by composing small widgets into a tree — the core idea you'll use all day.

Powered by Dart

A clean, typed language built for UIs, compiled ahead-of-time to fast native code.

Hot reload

See code changes in under a second without losing app state — a huge productivity boost.

Flutter Architecture

Three layers — your Dart code sits on top; the engine does the drawing.

Framework (Dart)

Material & Cupertino widgets • rendering, animation, gestures • your app code

Widgets

Rendering

Foundation

Engine (C/C++)

Skia/Impeller graphics • Dart runtime • text layout • low-level paint

Graphics

Dart VM

Text

Embedder (per-platform)

Hosts Flutter on Android, iOS, web, desktop • input, surface, lifecycle

Android

iOS

Web / Desktop

Your app *lives in the Framework layer. You rarely touch the engine or embedder — Flutter handles them for you.*

Drawing the UI & the Hot-Reload Workflow

How Flutter turns your code into pixels

1 Widget tree

You describe the UI as nested widgets (config).

2 Element tree

Flutter builds & diffs the live element tree.

3 Render tree

Layout + size each box; decide what to paint.

4 Engine paints

Skia / Impeller draws the frame to the screen.

The workflow superpower

Hot Reload

< 1 second

- Injects changed code into the running app
- Keeps current state (you stay on the same screen)
- Use constantly while tweaking UI

Hot Restart

a few seconds

- Restarts the app from scratch — state is lost
- Needed after changing main(), globals or state classes
- Still far faster than a native rebuild

Rule: *reach for hot reload first; hot restart only when reload says it can't apply the change.*

Flutter: Advantages, Limitations & What You Can Build

Advantages

- Single codebase → Android + iOS (and web/desktop)
- Fast development with hot reload
- Consistent UI — Flutter draws every pixel
- Great performance (compiled to native)
- Rich widget library: Material & Cupertino
- Strong community, backed by Google

Limitations

- Larger app size than pure native
- New platform widgets can arrive late
- Dart is less common than JS/Kotlin
- Very heavy 3D / games still favour native
- Some deep-platform features need plugins

What you can build

- Business & productivity apps
- E-commerce and delivery apps
- Social and content apps
- Dashboards and internal tools
- MVPs and startup products
- TaskFlow — our case study today

Dart Basics 1 — Variables, Types & Operators

Dart is statically typed but friendly: annotate types, or let var infer them.

main.dart

```dart
void main() {
  String name = 'Flutter';    // text
  int year = 2026;            // whole number
  double rating = 4.8;        // decimal
  bool isStable = true;       // true / false
  var language = 'Dart';      // type inferred
  const pi = 3.14;            // compile-time const

  // string interpolation with $
  print('$name in $year');
  print('Rating: $rating, stable: $isStable');

  // operators
  int total = year + 4;       // 2030
  bool good = rating > 4.0;   // true
}
```

Core types

- String — text in quotes
- int / double — numbers
- bool — true or false
- var — infer from value
- const / final — unchanging

Remember

- Use $var or ${expr} inside strings
- final = set once at runtime
- const = fixed at compile time
- Prefer explicit types in public APIs

Dart Basics 2 — Control Flow & Functions

control_flow.dart

```dart
// conditions
int score = 72;
if (score >= 50) {
  print('Pass');
} else {
  print('Retry');
}

// classic for loop
for (int i = 1; i <= 3; i++) {
  print('Task $i');
}

// for-in over a list
final tasks = ['Study', 'Code', 'Test'];
for (final t in tasks) {
  print(t);
}
```

functions.dart

```dart
// simple typed function
int add(int a, int b) {
  return a + b;
}

// arrow (one-line) function
int square(int n) => n * n;

// named + default parameters
String greet(String name,
    {String title = 'Student'}) {
  return 'Hello $title $name';
}

void main() {
  print(add(2, 3));         // 5
  print(greet('Sara'));     // Hello Student Sara
}
```

Dart Basics 3 — Classes & Objects

Dart is object-oriented — and your Flutter widgets are just classes.

task.dart

```dart
class Task {
  final String title;   // set once
  bool done;            // can change

  // constructor with named param + default
  Task(this.title, {this.done = false});

  // method
  void toggle() => done = !done;
}

void main() {
  final t = Task('Read chapter 9');
  print(t.done);   // false
  t.toggle();
  print(t.done);   // true
}
```

Anatomy of a class

- Fields — the data it holds
- Constructor — how you create one
- Methods — what it can do
- this.title — shortcut to assign a field

Why this matters for Flutter

- Every widget is a class
- A screen = objects composed together
- Your data models (like Task) are classes
- You'll subclass StatelessWidget next

Modern Dart — Null Safety & Async

Null Safety

Types are non-null by default — fewer crashes.

null_safety.dart

```dart
String name = 'Sara';    // never null
String? nickname;        // may be null

// safe access + default
int len = nickname?.length ?? 0;
print(len);    // 0

nickname = 'Sash';
// '!' asserts it is not null now
print(nickname!.length);  // 4
```

Async (Future / await)

Do slow work without freezing the UI.

async.dart

```dart
Future<String> loadTasks() async {
  await Future.delayed(
      Duration(seconds: 2));
  return 'Tasks loaded';
}

void main() async {
  print('Loading...');
  final msg = await loadTasks();
  print(msg);    // after 2s
}
```

? may be null **??** default if null **!** trust me, not null **async/await** wait without blocking

Checkpoint 2 — Read the Dart Code

Trace each snippet on paper. What does it print — and why? (3 minutes, then we compare.)

A

```dart
int add(int a, int b) => a + b;
void main() {
  print(add(4, 5));
}
```

B

```dart
String? city;
int n = city?.length ?? -1;
print(n);
```

C

```dart
final t = Task('Email', done: false);
t.toggle();
print(t.done);
// class Task { ... toggle() => done = !done; }
```

D

```dart
var nums = [1, 2, 3];
var sum = 0;
for (final x in nums) { sum += x; }
print(sum);
```

PART 4 OF 6

Building User Interfaces with Widgets



- Setting up the SDK and running your first app
- Project structure and the “everything is a widget” model
- Stateless vs stateful widgets and the app skeleton
- Core widgets, layouts, lists and reusable components

Setting Up & Running Flutter

One-time setup

1 **Install the Flutter SDK**

Download from flutter.dev; add it to your PATH.

2 **Install an editor**

VS Code or Android Studio + the Flutter / Dart plugins.

3 **Set up a device**

Android emulator, iOS simulator (Mac) or a real phone.

4 **Verify the toolchain**

Run flutter doctor and fix every ✗ it reports.

Everyday commands

terminal

```
flutter doctor      # check setup
flutter create app  # new project
cd app
flutter run         # build & launch
r  = hot reload   R = hot restart
q  = quit the app
```

If something breaks

- flutter doctor → read the ✗ lines first
- No device? start an emulator/simulator
- Weird errors? flutter clean, then run again

Flutter Project Structure

What flutter create gives you — and how to grow it sensibly.

Key files & folders

`lib/` All your Dart code lives here.

`lib/main.dart` Entry point — runApp() starts the app.

`pubspec.yaml` Dependencies, assets, fonts — the project manifest.

`android/` `ios/` Native project wrappers (rarely edited).

`test/` Automated tests for your app.

A scalable folder layout

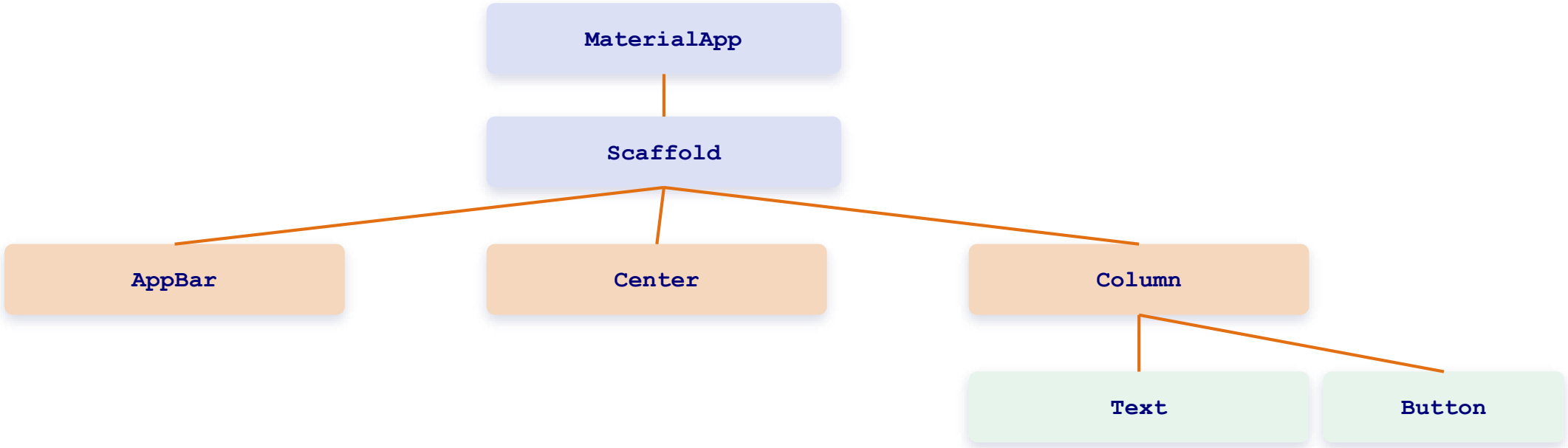
lib/

```
lib/
  main.dart           app entry
  models/             data classes
    task.dart
  screens/            full pages
    home_screen.dart
    add_task_screen.dart
  widgets/            reusable UI
    task_tile.dart
  services/           data / API
    task_repository.dart
```

Everything Is a Widget

The core idea. In Flutter, every part of the UI — text, a button, padding, even the whole screen and the app itself — is a widget. You build screens by composing small widgets into a tree.

A simple widget tree



Parent widgets hold child widgets — nesting builds the screen.

Stateless vs Stateful Widgets

Two kinds of widgets — the difference is whether they can change over time.

StatelessWidget

Never changes after it is built

- Draws once from the data it is given
- No internal mutable state
- Rebuilds only if its inputs change
- Lightweight and simple

Use for: labels, icons, a static task card, layout.

Example: TaskCard showing a title

StatefulWidget

Can rebuild itself when data changes

- Holds mutable state in a State object
- Call setState() to update the UI
- Rebuilds in response to interaction
- Slightly more code than stateless

Use for: checkboxes, forms, counters, toggles.

Example: a Task that can be ticked done

The App Skeleton — MaterialApp, Scaffold, AppBar

main.dart

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'TaskFlow',
      home: Scaffold(
        appBar: AppBar(title: Text('TaskFlow')),
        body: Center(child: Text('Hello, Flutter!')),
      ),
    );
  }
}
```

`runApp()`

Boots the app with your root widget.

`MaterialApp`

Wraps the app; themes, routes, title.

`Scaffold`

Standard screen: app bar + body.

`AppBar`

The top bar with a title.

`body`

The main content area of the screen.

`build()`

Returns the widget tree to display.

Core UI Widgets You'll Use Constantly



Text

Display a string, styled with TextStyle.



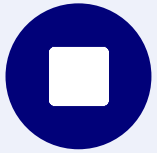
Image

Show assets or network images.



Buttons

ElevatedButton, TextButton, IconButton.



Container

A box: size, padding, colour, decoration.



Card

Rounded, elevated surface for grouping.



Padding

Adds space around a child widget.



Icon

Built-in Material icons via Icons.*



ListTile

Ready-made row: leading, title, trailing.

Layout Widgets — Row, Column & Stack

Arrange children horizontally, vertically, or layered on top of each other.

layout.dart

```dart
Column(
  crossAxisAlignment:
      CrossAxisAlignment.start,
  children: [
    Text('TaskFlow',
      style: TextStyle(fontSize: 22)),
    SizedBox(height: 8),
    Row(
      children: [
        Icon(Icons.check_circle),
        SizedBox(width: 6),
        Text('3 of 5 done'),
      ],
    ),
  ],
)
```

Column ↓

Stacks children top-to-bottom (the main axis is vertical).

Row →

Places children left-to-right (the main axis is horizontal).

Stack ⊕

Layers children on top of each other (e.g. a badge on an avatar).

Showing Many Items — ListView & GridView

Turn a list of data into a scrollable list or grid of widgets.

list_view.dart

```dart
final tasks = ['Study', 'Code', 'Test'];

ListView.builder(
  itemCount: tasks.length,
  itemBuilder: (context, index) {
    return ListTile(
      leading: Icon(Icons.task_alt),
      title: Text(tasks[index]),
    );
  },
)
```

grid_view.dart

```dart
GridView.count(
  crossAxisCount: 2,
  children: [
    Card(child: Center(
      child: Text('Today'))),
    Card(child: Center(
      child: Text('Upcoming'))),
    Card(child: Center(
      child: Text('Done'))),
  ],
)
```

ListView.builder *is efficient — it builds only the rows currently on screen, so long lists stay fast.*

Input & Flexible Sizing — TextField, Expanded, Flexible

input_layout.dart

```dart
// 1) text input
TextField(
  decoration: InputDecoration(
    labelText: 'New task',
    border: OutlineInputBorder(),
  ),
  onSubmitted: (value) {
    print('Add: $value');
  },
),

// 2) share space in a Row
Row(children: [
  Expanded(child: TextField()),
  Icon(Icons.add),
])
```

TextField

- InputDecoration adds label + border
- onChanged / onSubmitted handle input
- Pair with a TextEditingController to read text

Expanded

- Fills the remaining space in a Row/Column
- Fixes the dreaded overflow error
- Several Expanded → share space by flex
- Great for a wide input beside a button

Flexible

- Like Expanded but may take less space
- Use when a child can shrink but needn't fill
- Expanded = Flexible(fit: tight)

Building Reusable Custom Widgets

Why. Extract repeated UI into your own widget class — write it once, reuse it everywhere, and keep build methods short and readable. This is composition: small widgets combined into bigger ones.

task_tile.dart

```
class TaskTile extends StatelessWidget {
  final Task task;
  final VoidCallback onToggle;

  const TaskTile({super.key,
    required this.task, required this.onToggle});

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: Icon(task.done
        ? Icons.check_circle
        : Icons.circle_outlined),
      title: Text(task.title),
      onTap: onToggle,
    );
  }
}
```

How to use it

```
TaskTile(
  task: myTask,
  onToggle: () {},
)
```

Benefits

- No copy-paste — one source of truth
- Pass data in via constructor fields
- Pass actions in via callbacks
- Shorter, clearer parent widgets

Activity — Build the Widget Tree

Your task (8 minutes, in pairs). Sketch the widget tree for a single task row in TaskFlow: a card containing a checkbox icon on the left, the task title in the middle, and a delete icon on the right. Name every widget you would use.

Hints

- Start from a Card as the outer container
- A Row places the three parts side by side
- Use Icon for the checkbox and delete
- Wrap the title in Expanded so it fills space
- Add Padding / SizedBox for breathing room
- Which parts could change → stateful?

Deliverable

- A labelled tree diagram (boxes + lines)
- Parent → child relationships shown clearly
- Each node names a real Flutter widget
- One sentence: stateless or stateful, and why

We'll sketch one solution together on the board afterwards.

PART 5 OF 6

State, Interaction & Navigation



- Managing change with setState and handling events
- Forms, validation and the limits of setState
- Moving between screens with the Navigator
- Named routes and passing data between screens

State & Interaction — setState

State = data that can change while the app runs. Update it inside setState(), and Flutter rebuilds the widget to show the new value.

counter.dart

```dart
class Counter extends StatefulWidget {
  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0;            // the state

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () => setState(() => count++),
      child: Text('Done: $count'),
    );
  }
}
```

The update loop

1 User acts
taps the button (an event)

2 setState()
you change count inside it

3 Rebuild
build() runs again automatically

4 UI updates
new value appears on screen

Forms, Validation & the Limits of setState

form.dart

```dart
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    decoration: InputDecoration(
      labelText: 'Task title'),
    validator: (value) =>
      (value == null || value.isEmpty)
        ? 'Title is required'
        : null,
  ),
)

// on submit:
if (_formKey.currentState!.validate()) {
  // all fields valid → save
}
```

Form validation

- Wrap fields in a Form with a GlobalKey
- Each field gets a validator function
- Return an error string, or null if OK
- validate() checks them all at once

When setState isn’t enough

- Fine for state inside one widget
- Painful when many screens share data
- Then reach for state management:
- **Provider • Riverpod • BLoC**
- (Covered further in Lecture 10)

Navigation — Moving Between Screens

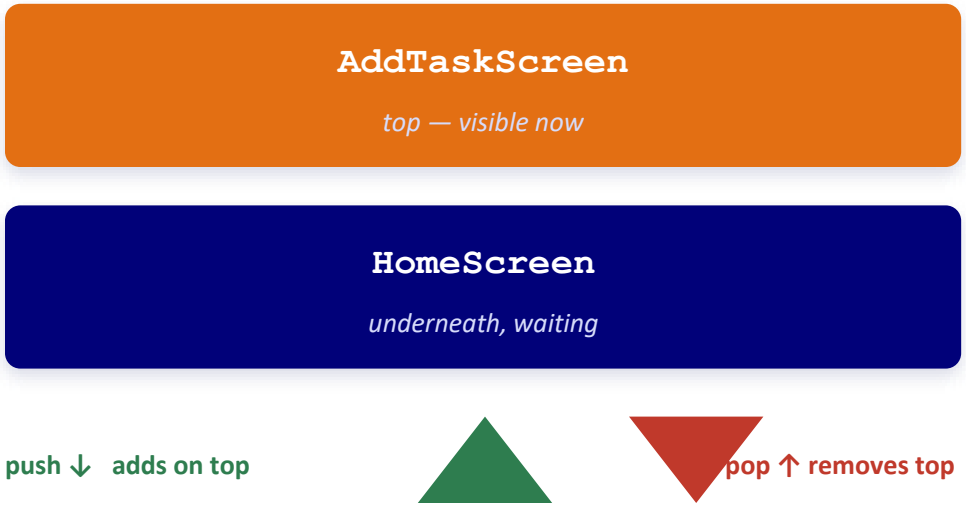
The navigator is a stack. push a new screen on top to go forward; pop the top screen off to go back — just like a stack of cards.

navigation.dart

```dart
// go to a new screen
ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => AddTaskScreen(),
      ),
    );
  },
  child: Text('Add task'),
)

// go back (e.g. an AppBar back button)
Navigator.pop(context);
```

The navigation stack



Flutter draws the back arrow automatically when you push a new route.

Named Routes & Passing Data

Name your screens in one place, then navigate by name — and pass data both ways.

routes.dart

```dart
MaterialApp(
  initialRoute: '/',
  routes: {
    '/':    (context) => HomeScreen(),
    '/add': (context) => AddTaskScreen(),
  },
)

// navigate by name
Navigator.pushNamed(context, '/add');
```

pass_data.dart

```dart
// HOME: open add screen, wait
final newTask = await
  Navigator.pushNamed(context, '/add');
// use the returned value
print('Got: $newTask');

// ADD SCREEN: send a result back
Navigator.pop(context, 'Buy groceries');
```

Two directions: pushNamed sends you forward (and can carry arguments); pop(context, value) sends a result back to the screen that opened it.

Checkpoint 3 — Stateless or Stateful?



For each widget, decide: StatelessWidget or StatefulWidget? Be ready to justify in one line. (3 minutes.)

1	An app logo shown in the header	*hint: never changes*
2	A checkbox that toggles a task done / not done	*hint: changes on tap*
3	A label that displays a task's title text	*hint: just displays*
4	A login form that validates and shows errors	*hint: holds input*
5	A counter showing how many tasks are completed	*hint: value changes*
6	An icon button that simply navigates to another screen	*hint: no own data*

PART 6 OF 6

Mobile Architecture & Case Study



- Why architecture matters: separation of concerns
- Layered and feature-based architectures for mobile
- Case study: designing the TaskFlow app
- Building TaskFlow — model, UI, state and navigation

Why Architecture Matters

Separation of concerns. Keep UI, business logic and data in separate, well-named places. As an app grows, this is what keeps it understandable, testable and changeable.



Everything in one file

- UI, logic and data tangled together
- Hard to find or change anything
- Almost impossible to test in isolation
- Breaks easily — one change ripples everywhere
- Two developers constantly collide



Clear architecture

- Each layer has one clear responsibility
- Easy to locate and modify code
- Layers can be tested independently
- Changes stay local — fewer surprises
- Teams work in parallel without clashing

Layered Mobile Architecture

A common, beginner-friendly structure — each layer talks only to the one below it.

UI Layer (Widgets / Screens)

What the user sees and taps — screens, widgets, layout.

HomeScreen, TaskTile

Service / Logic Layer

App rules and coordination — the 'what should happen'.

TaskService: add, complete, filter

Repository / Data Layer

Gets and saves data — hides where it comes from.

TaskRepository: local DB / API

Model Layer (Data classes)

The shape of your data — plain Dart objects.

class Task { title; done; }

Data flows down to fetch, and back up to display. *The UI never talks to the database directly.*

Feature-Based Structure & Knowing When to Scale

Organise by feature, not just by type

lib/features/

```
lib/
  features/
    tasks/
      task.dart         model
      task_service.dart logic
      home_screen.dart  UI
      task_tile.dart    UI
    auth/
      login_screen.dart
  shared/
    widgets/  utils/
```

Each feature is self-contained — easy to find, easy to grow.

Match structure to project size

Student / small app

- A few folders: models, screens, widgets
- setState for state — keep it simple
- Goal: clear and working, not perfect

Production / large app

- Feature folders + clear layer separation
- Dedicated state management (Provider/BLoC)
- Testing, CI/CD, error handling, logging
- Multiple developers, long lifespan

Case Study — Designing TaskFlow

TaskFlow — a simple task manager. You can see your tasks, add a new one, and tick tasks as done. Small enough to build today, but it exercises lists, forms, state and navigation together.

What it does



View tasks

A scrolling list of all tasks.



Add a task

A form to enter a new task.



Complete

Tap to mark a task done.

Screen flow



push → add → pop with the new task

Building TaskFlow (1) — Model & Task List

Step 1: the data model and a home screen that lists tasks.

task.dart + models

```dart
class Task {
  final String title;
  bool done;
  Task(this.title, {this.done = false});
}

// seed data
final tasks = <Task>[
  Task('Read chapter 9'),
  Task('Build TaskFlow'),
  Task('Submit lab', done: true),
];
```

home_screen.dart

```dart
Scaffold(
  appBar: AppBar(title: Text('TaskFlow')),
  body: ListView.builder(
    itemCount: tasks.length,
    itemBuilder: (context, i) {
      final t = tasks[i];
      return ListTile(
        leading: Icon(t.done
          ? Icons.check_circle
          : Icons.circle_outlined),
        title: Text(t.title),
      );
    },
  ),
)
```

Building TaskFlow (2) — Add Task & State

Step 2: a form to add tasks, with validation, updating the list via setState.

add_task_screen.dart

```dart
final _controller = TextEditingController();
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    controller: _controller,
    decoration: InputDecoration(
      labelText: 'Task title'),
    validator: (v) => (v == null || v.isEmpty)
      ? 'Please enter a title' : null,
  ),
)
```

on save (stateful)

```dart
void _save() {
  if (_formKey.currentState!.validate()) {
    setState(() {
      tasks.add(Task(_controller.text));
    });
    Navigator.pop(context);
  }
}

// completing a task, back on Home:
onTap: () => setState(
  () => t.done = !t.done),
```

Building TaskFlow (3) — Navigation & Running It

Step 3: wire the screens together, then run the same code on Android and iOS.

wiring it up

```
// Home: a button to open the add screen
floatingActionButton: FloatingActionButton(
  child: Icon(Icons.add),
  onPressed: () {
    Navigator.push(context,
      MaterialPageRoute(builder: (_) =>
        AddTaskScreen()));
  },
),

// Add screen: save button calls _save()
ElevatedButton(onPressed: _save,
  child: Text('Save task')),
```

TaskFlow now lists, adds and completes tasks across both platforms.

One codebase, both platforms

Android

flutter run on an emulator or device — Material look by default.

iOS

Same Dart code on the simulator (Mac) — no rewrite needed.

That's the win. One codebase → two native apps.

Activity — Design Your App's Screens & Folders

Your task (10 minutes, in pairs). Pick a simple app idea (e.g. a notes app, expense tracker, or habit tracker). Design its screens, its data model, and a sensible folder structure — applying today's architecture ideas.

Deliver three things

- **1. Screen list**
- Name each screen and how you move between them
- **2. Data model**
- One or two Dart classes (fields only)
- **3. Folder structure**
- models / screens / widgets / services

Checklist

- Which screens are stateful vs stateless?
- What does each Navigator route lead to?
- Where does data live (which layer)?
- Is the UI kept separate from the data?
- Could a teammate find things easily?
- Is it as simple as it can be?

Sketch it on paper — a few volunteers will share at the start of next session.

Common Beginner Mistakes in Flutter

Recognise these early — they trip up almost everyone at first.

Changing state outside setState()

Fix: Wrap every state change in setState() so the UI rebuilds.

Overflow errors in Row / Column

Fix: Wrap flexible children in Expanded or Flexible.

Using Stateful when Stateless will do

Fix: Default to Stateless; go Stateful only when data changes.

Everything piled into main.dart

Fix: Split into models, screens, widgets and services.

Forgetting to handle null

Fix: Use ?, ?? and ! deliberately — lean on null safety.

Building all list rows at once

Fix: Use ListView.builder for long or dynamic lists.

ASSESSMENT

Quick Quiz — Questions 1–5

Choose the best answer. We'll review them together — answers are on the next part of the session.

1. Which approach uses ONE codebase to target both Android and iOS?

(A) Native (B) Cross-platform (C) Two native apps (D) Assembly

2. Which programming language does Flutter use?

(A) JavaScript (B) Kotlin (C) Dart (D) Swift

3. In Flutter, "everything is a …"

(A) function (B) widget (C) file (D) service

4. Which call triggers a UI rebuild in a StatefulWidget?

(A) build() (B) rebuild() (C) setState() (D) refresh()

5. Which widget arranges its children horizontally?

(A) Column (B) Row (C) Stack (D) ListView

Quick Quiz — Questions 6–10

Last five — same rules. Jot your answers; we'll go through the full key together.

6. A StatelessWidget is the best choice for …

(A) a checkbox (B) a validated form (C) a static label (D) a live counter

7. Which call navigates FORWARD to a new screen?

(A) Navigator.pop (B) Navigator.push (C) setState (D) runApp

8. Where does a Flutter app start executing?

(A) pubspec.yaml (B) android/ (C) lib/main.dart (D) test/

9. In layered architecture, the UI layer should …

(A) talk to the DB directly (B) go through a service/repository (C) hold all logic (D) be skipped

10. While applying code changes, hot reload keeps …

(A) nothing (B) the current app state (C) only the AppBar (D) the file names

Summary — What We Covered Today

Part 1 • Mobile foundations

Native, cross-platform, hybrid and PWA — and how teams choose.

Part 2 • Cross-platform & Flutter

One codebase, Flutter's architecture, advantages and limits.

Part 3 • The Dart language

Variables, control flow, classes, null safety and async.

Part 4 • Building UIs with widgets

Everything is a widget; layouts, lists and custom widgets.

Part 5 • State & navigation

setState, forms, and moving between screens.

Part 6 • Architecture & TaskFlow

Layered design, and building the case-study app end to end.

From "why mobile?" to a working cross-platform app — you now have the full beginner toolkit.

Learning Outcomes, Lab & What's Next

How today maps to the module outcomes

LO	Outcome — covered today
LO1	Evaluate mobile frameworks: compared native / cross-platform / hybrid / PWA.
LO2	Apply a framework: built TaskFlow with Flutter UI, state and navigation.
LO4	Select & justify: chose approaches against requirements, team and timeline.

LO3 (integration, CI/CD, quality, deployment) is developed in later lectures and the project.

In the lab

- Install Flutter and run flutter doctor
- Build and run TaskFlow yourself
- Extend it: add delete, or a task count

Next — Lecture 10

- Richer Flutter UI and device features
- Working with data and local storage
- State management beyond setState
- Prep: revise widgets, state & navigation

RESOURCES

References & Further Reading

Use these to go deeper and to support your lab work and project.

Core resources

- Rose, R. (2022) Flutter and Dart Cookbook. Sebastopol, CA: O'Reilly Media.
- Flutter official documentation — flutter.dev/docs (widgets, layout, navigation, cookbook).
- Dart language tour — dart.dev/guides (syntax, null safety, async).

Tools & packages

- pub.dev — the Dart & Flutter package registry
- Flutter widget catalog — docs.flutter.dev/ui/widgets
- DartPad — dartpad.dev (run Dart in the browser)
- Flutter DevTools — debugging & performance

Module & supplementary

- SE3090 module outline — LMS (outcomes & schedule)
- Lecture slides & lab sheets — course LMS
- Banks, A. & Porcello, E. (2020) Learning React. O'Reilly.
- Google Codelabs — 'Write your first Flutter app'

Always prefer the official flutter.dev and dart.dev docs — they are current, complete and free.